

Prototypes and prototypical inheritance

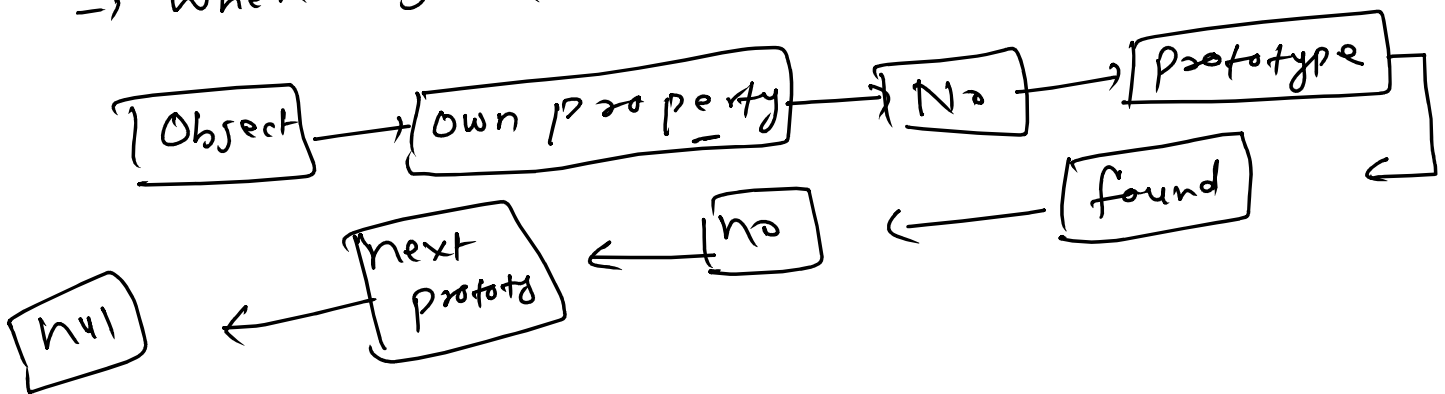
Prototypes → (— proto —)

→ Every JS object can have a prototype

+ A prototype is another object from which an object can inherit properties and methods.

Prototype chain →

→ When JS looks for a property or methods



(Object name. — proto —) → Prototypes
 ↓
object

⇒ Object.getPrototypeOf(—) → ✓
 ⇒ Object.setPrototypeOf(—) →
 ⇒ Objectname.hasOwnProperty(—) →

```

// const user = {
//   name : "Satyam"
// }
// console.log(user)
// console.log(user.toString())
// console.log(user.meraname()) //error
// console.log(user.__proto__)
// const user = {
//   name : "Satyam"
// }
// console.log(Object.getPrototypeOf(user))
// console.log(user.hasOwnProperty("toString"))
  
```

```

//prototypical inheritance
// const Person = {
//   greet(){
//     console.log("Hello, i'm from person")
//   }
// }
// const student = Object.create(Person)
// student.study = function (){
//   console.log("Hello, i'm from student")
// }
// student.greet()
  
```

```

// prototype and __proto__
// prototype => function and classes
// __proto__ => exists on all object
// class Person{
//   constructor(name){
//     this.name = name
//   }
  
```

```
// }
// console.log(Person.prototype)
// Person.prototype.sayHi = function () {
//     console.log("Hi, im sayHi method")
// }
// console.log(Person.prototype)
// const student1 = new Person("Satyam")
// console.log(student1.__proto__ === Person.prototype)

// class and prototypes
// class User {
//     constructor (name){
//         this.name = name
//     }
//     greet(){
//         console.log("greet method")
//     }
// }
// const user = new User("satyam")
// console.log(User.prototype)
// user.greet()
// class User => User.prototype => greet()
```